AI Usage Note — RootLens

Post-Incident RCA Drafter · Infinite AI Prototype Challenge

---

1. AI Tools Used

Tool / Platform	Primary Use
Claude (claude.ai)	Problem selection, RCA prompt design, debugging, demo scripting
ChatGPT	Cross-checking ideas, alternative approaches, quick lookups
Antigravity IDE — Claude Opus 4.6	Main coding environment — backend scaffolding, React components, debugging
Gemini 3.1 Pro	Agent prompt refinement, architecture planning, complex reasoning
Gemini 3.1 Flash	Fast iterations, quick code generation, rapid UI changes
Stitch MCP	Data pipeline integration, connecting external sources to the RCA workflow
Insforge MCP	Historical incident search, context enrichment for the RCA agent pipeline

2. What AI Helped With

Problem Selection

- Ranked all five problem statements using Claude — got back **feasibility scores, demo potential,** and predictions on which problems other teams would attempt.
- Analysis pushed us toward **PS-05 (RCA Drafter)** and flagged SD-07 / SD-08 as low-impact traps.

Backend Scaffolding — Antigravity IDE (Claude Opus 4.6)

- Generated full **FastAPI skeleton** — main.py, database.py, models.py, schemas.py
- SQLAlchemy tables, all **five agent files**, MCP layer (inforge_client.py, tools.py), and Pydantic schemas
- **Saved 4–5 hours** of repetitive setup. Logic inside agents still needed our input.

Frontend

- **Glassmorphism design system** — described the look (dark bg, blurred glass panels) and AI generated the CSS in one pass.
- Built all **seven React components** and the Material 3 bento-grid dashboard layout.

RCA System Prompt

- Iterated across **Claude, Gemini 3.1 Pro, and Gemini 3.1 Flash** to land on a strict **seven-section SRE persona prompt** that gave consistent, parseable JSON output.

MCP Integration

- **Insforge MCP** pulled historical incidents into the RCA context before the final report was generated.
- **Stitch MCP** connected external data sources into the investigation pipeline.

Debugging

- Pipeline was silently returning empty results. Claude diagnosed **Groq's free tier blocking cloud IPs**, suggested switching to OpenRouter, gave the exact three lines to change. Fixed in under 10 minutes.

3. What AI Got Wrong

Hallucinated GitHub Links

- Gave three specific GitHub URLs — all wrong or completely made up. **vstorm-co/full-stack-ai-agent-template** did not exist. Also recommended a fake npm package (fastapi-fullstack).
- **Lesson:** Always open the link in a browser before trusting it.

Broken FastAPI Syntax

- Generated **@post(...)** instead of **@app.post(...)** — would crash on startup. AI wrote it without the surrounding file context.
- **Lesson:** Always paste the surrounding file context with code generation requests.

Silent Groq Failure

- Recommended Groq without mentioning it blocks cloud datacenter IPs on the free tier. Every agent returned empty results silently — no error at all.
- **Lesson:** Ask explicitly about deployment and hosting limitations upfront.

Incomplete MCP Wiring

- PRD specified MCP integration but **tools.py was empty** in the generated files. Orchestrator never called historical search. Caught only through a manual line-by-line audit.
- **Lesson:** Treat the spec and the generated code as two separate things. Always verify they match.

Hardcoded Values & Git Conflicts

- Initial components had **hardcoded API URLs** instead of env vars. Had to replace manually.
- Suggested **git pull --rebase** caused merge conflicts in .gitignore. Had to resolve manually.

4. Best Prompts Used

1. Competitive Framing

"This is a 1-2 day sprint competing against 13 teams. Tell me which project to pick and exactly how to build it to maximise the probability of winning."

- **Why it worked:** Added competitive context — got judge psychology analysis and ranked demo impact instead of a neutral breakdown.

2. No Rewrites When Adding Features

"Analyse the entire codebase before making any changes. Do not modify, rename, or delete any existing folders. Write an implementation plan first. Then implement."

- **Why it worked:** Prevents coding agents from regenerating everything from scratch and breaking existing files.

3. No Inline Comments

"Generate only production-ready code. Do not include any inline comments, # notes, or explanatory text inside the files."

- **Why it worked:** Strips padding, forces tighter code, makes logic errors easier to spot.

4. Visual Vocabulary for UI

"Build a split-pane workspace. Left panel: raw logs. Right panel: AI report. Use translucent glass cards with backdrop blur, subtle borders, deep navy background."

- **Why it worked:** Naming specific visual elements (backdrop blur, split pane) instead of "make it modern" produced a working component in one pass.

5 — Targeted Bug Fix

"Fix the issue where InvestigationView shows "No Investigation Selected". DO NOT rewrite any files. Only fix the specific broken connection. ANALYZE FIRST — do not change anything yet."

- **Why it worked:** The constraint "ANALYZE FIRST" stopped the model from jumping to a full rewrite. The actual fix was one line.

6 — SRE Agent Persona

"You are a senior SRE with 10+ years in distributed systems. Parse the raw log text and extract every significant event. Output ONLY valid JSON — no preamble, no markdown, no commentary."

- **Why it worked:** Professional persona anchors output style. JSON-only requirement made log agent output consistently parseable.

5. Key Takeaways

- **Specificity beats vagueness.** Exact file paths, visual names, and explicit constraints consistently outperformed vague requests.
- **AI is good at structure, weak at wiring.** Scaffolding is reliable. End-to-end integration (MCP, routing, context flow) needs manual verification.
- **Never trust a URL without clicking it.** Hallucinated packages and repos are indistinguishable from real ones in AI output.
- **Multiple models helped.** Cross-checking between Claude, Gemini 3.1 Pro, Gemini 3.1 Flash, and ChatGPT caught issues no single model surfaced.
- **Incremental prompts beat bulk generation.** Small targeted prompts with clear acceptance criteria produced cleaner, more reliable output.
- **Net time saved: ~8–10 hours** on scaffolding, styling, and debugging. Lost ~2–3 hours to hallucinated packages, silent failures, and incomplete wiring. Manual audits made the difference.
